**EAST WEST UNIVERSITY**

**Department of Computer Science and Engineering**

**BSc in Computer Science and Engineering**

**Midterm Examination, Fall 2025**

**Course:** CSE207 Data Structures, Section-6 **Date:** November 16, 2025
**Instructor:** Dr. Nafis Sadeq, Assistant Professor, CSE Department
**Full Marks:** 30
**Time:** 1 Hour and 20 Minutes

**Note:** There are **SIX** questions. Answer **ALL** of them. Course Outcome (CO), Cognitive Level, and Mark of each question are mentioned at the right margin.

---

**1.** Consider the following sorted array:

$$A = [1, 3, 9, 12, 17, 21, 26, 30, 35]$$

Simulate the steps of a **binary search** to find the value **21**. Clearly show the low-index, high-index, mid-index, mid-value, and action taken at each step.

**[CO1, C3] [Mark: 5]**

**2.** The following C++ function attempts to insert a new node at the **tail** of a Singly Linked List:

```
void insertTail(Node&* head, int v) {
    Node* temp = new Node(v);
    if(head == NULL) head = temp;
    else {
        Node* cur = head;
        while(cur != NULL)
            cur = cur->next;
        cur->next = temp;
    }
}
```

(a) Identify any logical errors in this implementation. (b) Write the corrected function. **[CO1, C3] [Mark: 5]**

**3.** (a) Define a **stack** data structure. (b) Explain the operations: `push`, `pop`, `top`/`peek`, and `isEmpty`. (c) Mention one real-world application of stacks. **[CO1, C3] [Mark: 5]**

**4.** Convert the following infix expression into postfix form **using a stack**. For each step, show the operation, return value, updated stack and postfix string:

$$(a + b) * c/(d - e)$$

**[CO1, C3] [Mark: 5]**

**5.** Consider a **Queue implemented with a Circular Buffer** of size 6 (indices 0–5). Initially:

$$front = 2, \quad rear = 5$$

and the queue contains:

$$[_, _, 7, 15, 19, _]$$

Simulate the queue after the following operations, showing the return value, updated front, rear, and buffer contents after each step:

- `enqueue(8)`
- `enqueue(14)`
- `dequeue()`
- `enqueue(52)`

**[CO1, C3] [Mark: 5]**

**6.** (a) Write the recursive definition of the vanilla Fibonacci function. (b) Determine the time complexity of this naive recursive version using the function call tree and explain why it is inefficient. (c) Describe the memorization method to improve the time complexity. **[CO1, C3] [Mark: 5]**